Operating system programming

Indice

We are going to focus more on microcontrollers, since they are enough to get the point without the complexity of full CPUs.

Booting an OS

Processes in a general-purpose OS run in a virtual address space, which requires HW Support (MMU/TLB). Inside an OS we can access both physical memory and virtual memory. Most microcontroller do not even have an MMU so the code needs to work only with physical memory addresses (even for applications).

OS/application code is located in the nonvolatile Flash memory which is memory mapped. Stack, heap and global variables are placed in RAM. An address range is reserved for HW peripherals, but it is sparsely used and mostly unmapped. Parts of the address space is unmapped, accessing these areas causes a fault interrupt to be generated.

Linker scripts

With virtual memory, we decide the addresses by convention. Thus we can use one linker for all programs. With physical memory we must adapt to the memory layout, meaning we need a different linker for each hardware platform we support. Loadable application (if there are any) must be compiled as PIC.

When compiling an OS, we must tell the linker at which physical address to put our code/data. For micro controllers we usually have:

1. Code (`.text` section): Flash memory
2. Data (`.text`/`.bss` section): RAM
3. Stack and heap are allocated at runtime

Linker scripts configure the linker (such as GNU's `ld`).

```
ENTRY(Reset_Handler) /* the first function called at boot */

/* The MEMORY section specifies the hardware memory layout */
MEMORY
{
  flash(rx) : ORIGIN = 0x08000000, LENGTH = 1M
  ram(wx)   : ORIGIN = 0x20000000, LENGTH = 128K
}

/* _stack_top is a variable that defines the start of address of the stack.
 *
 * Variables will be added to the program's symbol table and can be accessed by
 * them
 */
_stack_top = 0x20000000+128*1024;

/* SECTIONS tells the linker how to map the program sections into memory regions
 * `.` is as special variable called the "location counter"; it is incremented
 * after each section mapping by the section size.
 */
SECTIONS
{
  . = 0;

  /* Map the .isr_vector, .text and .rodata sections to the flash.
   * `*` is a wildcard and means any executable name
   */
  .text :
  {
    KEEP(*(.isr_vector)) /* All executables' `.isr_vector` is placed first,
                          * `KEEP` prevents any garbage collection from omitting
                          *  it
                          */
    *(.text)      /* Then we place the `.text` */
    . = ALIGN(4); /* Align on the byte boundary (ABI and platform dependant) */
    *(.rodata)    /* Last we place the `.rodata` */
  } > flash /* Put this section to flash */

   /* Output section for `.data` */
  .data :
  {
    _data = .; /* Declare a variable holding the start of the `.data` section */
    *(.data)   /* Place the section */
    . = ALIGN(8); /* Align it */
    _edata = .; /* Declare a variable holding the end of the section */
  } > ram AT > flash /* Then load this section in RAM, however, since at boot
                      * the memory might me uninitialized, place a copy of it
                      * into flash so that the OS can initialized it at runtime
                      */
  _etext = LOADADDR(.data); /* Save the end of `.text` in a variable (LOADADDR
                             * indicates the address to which the section is
                             * loaded). In conjunction with the other two
                             * previous variables, we can copy the `.data`
                             * section from flash to memory
                             */
```

```
  /* `.bss` is uninitialized data that should be zeroed out at runtime. We,
   * again, save the start/end of it so we can zero-it at runtime
   *
   * The reason why we do it outside the output section is to include an
   * "anonymous" (`.common`) section that also needs zero-ing. Summary: it has
   * been this way since the 70's and fortran, don't ask questions.
   */
  _bss_start = .;
  .bss :
  {
    *(.bss)
    . = ALIGN(8);
  } > ram
  _bss_end = .;

  _end = .; /* End of application code, we can start the heap at this address */
}
```

Booting the damn thing

The place where the microcontroller/CPU start executing code is architecture dependant:

- On PC, it starts from the BIOS/UEFI that loads a second level bootloader (such as GRUB) into memory
- Microcontrollers have memory-mapped nonvolatile memory (the flash), meaning we can directly boot the kernel without needing to load it in RAM.

There are two ways to identify the first instruction to run:

1. Set the Program Counter to a predefined address (e.g., `0x0`) and start from there
2. Read a predefined memory location containing the address of the first instruction, and use that value to initialize the Program Counter (approach used by e.g. ARM Cortex)

Assuming an ARM Cortex-based STM32F407 micro, the address of the first instruction must be placed at `0x08000004` (flash memory base + `0x4`). Is it enough to put there the address of the first C function of our kernel there? No, we also need to ensure some of the guarantees about the execution environment that high-level languages (such as C) need to function. This part will obviously need to be written in assembly language.

Booting a C kernel

- The stack pointer register must point to the top of a suitable memory area
  - The compiler implicitly uses the stack to allocate local variables within functions
  - Until then, only assembler code can be executed
- Global and static initialized variables must set to their initial value
  - Since they are placed in RAM, and after the power on the content of RAM is random, they must be explicitly initialized
- Global and static uninitialized variables must be set to 0

Example startup code (ARM):

```
  .syntax unified  // since ARM is a family of different instruction sets, we
  .cpu cortex-m4   // need to specify for what CPU we are writing for
  .thumb

  // Section containing a table of pointers:
  // - 0x08000000 = stack pointer init
  // - 0x08000004 = program counter init
  // Additional entries (not shown) are addresses of interrupt handlers
  .section .isr_vector
  .global __Vectors // global variable
__Vectors:
```

```
  .word _stack_top  // defined in liker script
  .word Reset_Handler

  // Define the code of the first function executed by the kernel
  .section .text
  .global Reset_Handler          // Define it as a global function
  .type Reset_Handler, %function //
Reset_Handler:
  /* Copy .data from FLASH to RAM */
  ldr r0, =_data  // \
  ldr r1, =_edata // | From the linker script
  ldr r2, =_etext // /
  cmp r0, r1
  beq nodata
dataloop:
  ldr r3, [r2], #4
  str r3, [r0], #4
  cmp r1, r0
  bne dataloop
nodata:
  // Zero-out the bss
  ldr r0, =_bss_start // | From the linker script
  ldr r1, =_bss_end   // |
  cmp r0, r1
  beq nobss
  movs r3, #0
bssloop:
  str r3, [r0], #4
  cmp r1, r0
  bne bssloop
nobss:
  /* Jump to kernel C entry point */
  bl kernel_entry_point
  .size Reset_Handler, .-Reset_Handler
```

Booting a C++ kernel

C++ has the same requirements as C, but with some extras:

1. Constructors of global objects need to be called before main
2. If using C++ exception, additional sections are generated by the compiler that need to be linked
    - We are not going to see this and thus we cannot use exceptions

To ensure the first requirement, the compiler generates a section called `.init_array`, which is basically a table of function pointer to global object constructors. To link it we need to modify our linker script:

```
SECTIONS
{
  /* ... */
  .text :
  {
    /* ... */
    /* Table of global constructors, for C++ */
    . = ALIGN(4);
    __init_array_start = .;
    KEEP (*(.init_array))
    __init_array_end = .;
    } > flash
```

```
  }
}
```

For the C++ startup script, we just modify the part after `nobss` to initialize objects before jumping:

```
  // ...
nobss:
  /* Call global contructors for C++. Can't use r0-r3 as the callee
   * doesn't preserve them
   */
  ldr r4, =__init_array_start
  ldr r5, =__init_array_end
  cmp r4, r5
  beq noctor
ctorloop:
  ldr r3, [r4], #4
  blx r3
  cmp r5, r4
  bne ctorloop
noctor:
  /* Jump to C++ kernel entry point */
  bl kernel_entry_point
  .size Reset_Handler, .-Reset_Handler
```

Accessing peripherals

How do we access our peripherals? The most common way is peripheral registers, memory locations mapped to specific addresses in the processor address space. The addresses of this registers is always physical, also in micros (this is the most common reason to use physical addressing in kernel code).

Pheripherals registers are, in some ways, similar to plain old variables:

- Accessible in the same way (load/store)
- 8, 16, or 32 bit wide, like `unit8_t`, `uint16_t` and `uint32_t` data types in C

… but also fundamentally different:

- Writing to these registers causes actions int the real world
- They have well-specified memory addresses
- They are not exclusive to software, but shared between hardware and software
  - The hardware can decide to change the content of a register to signal events or status flags, while variables simply keep the value stored in them by the programmer

How can we know the peripherals and their register available in a given architecture? Read the documentation of the chip: for micros, all peripherals are on chip, meaning they are documented in the micros data sheet.

Example of peripheral access

```
// Assume that there is a 32-bit register called IODIR0 at address
// 0xE0028008

// ======================================== Clearing a register the simple way =
void clearReg() {
  (*((volatile unsigned int *) 0xe0028008)) = 0;
}

// ============================== Clearing a register the more readable way 1 =
// These defines are usually grouped in a header file that is usually provided
// by the manufacturer
```

```c
#define IODIR0 (*((volatile unsigned int *) 0xe0028008))

void clearReg() {
  IODIR0 = 0;
}

// ============================== Clearing a register the more readable way 2 =
// Usually a peripheral has more than one register. These register can be
// grouped into a struct and mapped to the peripheral base address (the lowest
// address used)
struct GpioRegs {
  volatile unsigned int CRL;
  volatile unsigned int CRH;
  unsigned int gap;
  volatile unsigned short BSRR;
  volatile unsigned short BRR;
};
#define GPIO ((GpioRegs*)0xfeeeab00)

void clearReg() {
  GPIO->CRL = 0;
}

// There is no difference between "clean method" 1 or 2. Just that some
// manufacturers use the first and some the second.
```

Bit manipulation of peripheral registers

Classification of bits in peripheral registers:

- Control bits:
  - Change the behavior of the peripheral
  - Usually readable/writable by software
  - Usually changed infrequently, only during "configuration"
- Status bits or flags:
  - Allow software to query peripheral status
  - Usually read-only or some advanced access permissions (see read-clear registers)
- Data bits:
  - Exchange data between software and hardware
  - Read/write, read-only or write-only depending on the expected data flow

A register can have flags that can be set by hardware and cleared by software. To clear a flag we can do `REG &= ~(1<<0)`. This is a read-modify-write operation. However, the hardware can concurrently modify other flags during our read-modify-write operation, meaning we have a race condition that can be solved by software. For this reason hardware provides read-clear registers:

1. `rc_w1`: read and clear by writing 1
   - Bit can be read, and cleared by writing 1. Writing 0 has no effect
2. `rc_w0`: read and clear by writing 0
   - The opposite of the first case

This means we can simply do one write `REG = (1<<0)` to clear registers.

Interrupts

The only way we know up to now to check whether some hardware condition happened is to poll status bits. This, however, has very good responsiveness but terrible efficiency if events are sporadic. Interrupts solve the some of the problems of polling: they can be thought as a mechanism to let hardware call a software function when an event occurs.

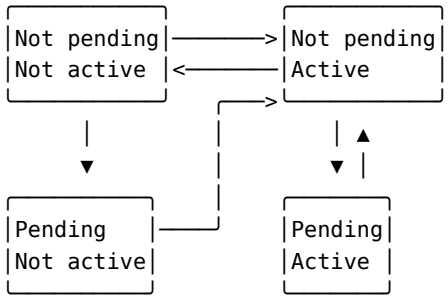
Interrupts can pause the normal code execution in between any two assembly instructions, and jump to a function, the "interrupt service routine (ISR)" (on certain architectures higher priority interrupts can even interrupt lower priority ones, called interrupt nesting). When the ISR completes, the processor reverts back to executing to normal code (or lower level interrupt, if nested). Important remarks:

1. Interrupts are always run to completion, an interrupt must NEVER block, or it will block all normal code (the entire OS + applications)
2. Interrupts should be written to be as fast as possible, in order to minimize the time interference with the main code

Interrupts require cooperation between three hardware components, not transparent to software (in a micro, all three inside the same chip):

- The CPU:
  - The CPU contains the logic for interrupt execution
  - It has a global bit to enable/disable all interrupts (with special instructions)
  - OSes provide locks that use this bit to protect critical sections
  - When peripherals request interrupt execution while interrupts are disabled, interrupt requests remain pending and will be executed when interrupts are enabled again
- The interrupt controller
  - The ARM interrupt controller has three bits for each peripheral: enable, active and pending
- The peripheral
  - Each event has an individual enable bit to let software decide which events should generate interrupts
  - Each of event has an event (status) bit, signaling that the event occurred to still enable polling when interrupts are disabled
  - A single line goes from the peripheral to the interrupt controller

For each interrupt source, the interrupt controller implements a state machine with the pending and active bits:



- If the peripheral triggers an interrupt when the machine is executing normal code with interrupts enable the state goes from (not pending, not active) to (not pending, active) and remains as such for the whole ISR duration
- When the CPU is done with the ISR we go back to (not pending, not active)
- If a peripheral sends an interrupt when interrupts are disable, it will be enqueued as (pending, not active); when interrupts are then re-enabled and the ISR be executed, the state will change to (no pending, active) and go about as seen previously
- If while executing the ISR the same peripheral sends another interrupt, the state goes to (pending, active)
  - When the ISR completes its execution, the same ISR is called again and the sate changes to (not pending, active)

Handling concurrency in Miosix

Every OS provides synchronization primitives for kernelspace code. Miosix does to, but supports standard C/C++ libraries and POSIX also in kernel (to allow application development in-kernel) meaning we can use standard C++11/POSIX primitives.

Code executed inside an interrupt service routine accessing variables shared with normal code in a single-core architecture requires no lock, the interrupt is the lock. Normal code accessing variables shared with an interrupt service routine must disable interrupts, Miosix provides these primitives:

1. `class FastInterruptDisableLock`: disables interrupts in the constructor and reenabes them in the destructor (you must be sure interrupts were enabled prior to calling this)
2. `class InterruptDisableLock`: conceptually equivalent to a `recursive_mutex`

Blocking normal code while waiting for an IRQ is conceptually equivalent to waiting for a `condition_variable`:

- All non-interrupt code in Miosix is an instance of the `class Thread` (basically the TCB of the kernel) which provides:
  - `Thread::IRQenableIrqAndWait(FastInterruptDisableLock&)`: atomically marks the thread as blocked, enables interrupts and schedules the next ready thread; must be called with interrupts disabled
  - `threadPtr->IRQwakeup()`: marks the thread as ready to be scheduled

Miosix also provides a "design pattern"-based approach to implement a producer-consumer with interrupts, in the form of a synchronized `Queue` class.

To be accessible to userspace applications, device drivers in Miosix need to be registered in `/dev` (like in other UNIX-likes). Device files are created by subclassing the `Device` class and providing implementations for needed functions (`readBlock`, `writeBlock` and `ioctl`). Devices are instantiated by adding an instance of the class to the devfs in the board support package.

Working with the linux kernel

Accessing peripherals

The Linux kernel lives in a virtual address space, however peripherals are located to physical address spaces. To interact with them we need to first request the region using `request_mem_region` and then map it into virtual space using `ioremap`. Then we can interact with it using regular load and stores.

For x86 systems, we might have to use port-based I/O, which uses a different API and ad-hoc machine instructions:

1. `request_region(peripheral_base_addr, peripheral_size, "drivername")`
2. `inb(addr)` / `inw(addr)` / `inl(addr)` to read
3. `outb(addr)` / `outw(addr)` / `outl(addr)` to write

Concurrency in the linux kernel

Linux does not support POSIX inside the kernel, has its own API:

1. `task_struct *t = kthread_run(threadfunc, arg, "threadname")` creates a kernel thread
2. `kthread_stop(t)`: join a kernel thread
3. `kthread_should_stop()`: must be called periodically in the kernel thread; calling `kthread_stop` from another thread makes it return true

Mutexes take the for of `struct mutex` (with `mutex_init`, `mutex_lock` and `mutex_unlock`). There is no support for the fancy scope-locking of cpp and for condition variables.

On multicore architectures a shared variable may be accessed by an interrupt on one core and by normal code on another. Since interrupts cannot be stopped anche re-scheduled, we need to use spinlocks:

1. `spin_lock_init`
2. `splin_lock` and `spin_unlock`: must be called from the interrupt side while accessing shared data
3. `spin_lock_irq` and `spin_unlock_irq`: must be called from the normal code side, also disables interrupts

To block code waiting for an interrupts we use `wait_queue_head_t`:

1. `init_waitqueue_head`: initializes the waitqueue
2. `wait_event_interruptible_lock_irq(queue, condition, spinlock)`: equivalent to condition variable wait; atomically unlocks the spinlock and blocks until the condition becomes true
3. `wake_up(&waitqueue)`: equivalent to `notify_one`, wake one thread from the waitqeue; can be called from an IRQ

Filesystem

Linux drivers are exposed in `/dev`. To register our driver we can do:

```
struct file_operations my_driver_fops = {
  .owner=THIS_MODULE,
  .write=my_driver_write,
  .read=my_driver_read,
};
//...
int major = register_chrdev(0, "name", &my_driver_fops);
```

While to unregister we use `unregister_chrdev(major, "name")`.

The registered r/w functions must have the following prototypes:

- `ssize_t write(struct file *f, const char __user *buf, size_t size, loff_t *o)`
- `ssize_t read(struct file *f, char __user *buf, size_t size, loff_t *o)`

There also other functions for e.g. `ioctl`. Note the `__user`: the linux kernel requires to explicitly copy data to and from userspace using:

- `unsigned copy_from_user(void *to, const void __user *from, unsigned n)`
- `unsigned copy_to_user(void __user *to, const void *from, unsigned n)`

The functions copy `n` bytes to/from userspace to/from kernelspace and return the number of bytes NOT copied (0 unless an error occurs). These functions also perform validation, so not using them poses a serious security risk.

To expose, then the driver in `/dev`, depending on the system (usually a daemon does it for us, but sometimes it might not be present), we need to create the special file by calling `mknod`.